DSCI 551 COURSE PROJECT, SPRING'26

Phase 3: Final Report

Group Members

Chanyoung Kim
Yogita Mutyala

USC Viterbi
School of Engineering

Introduction & Motivation

Financial institutions process millions of transactions daily. Detecting money laundering within that volume has a dual nature: transaction records and account metadata need the integrity guarantees of a relational database, while suspicious patterns like structured deposits and circular fund flows only become visible when tracing relationships across multiple accounts. Traditionally, this requires two separate systems, adding synchronization complexity and infrastructure cost.

This project investigates whether a single PostgreSQL instance extended with Apache AGE can serve both purposes. The application, Hybrid-AML, implements two AML detection rules on a synthetic financial dataset. The first targets smurfing, where a large sum is broken into sub-threshold cash deposits funneled into one account before being wired out. The second targets circular trading, where funds move in a closed loop between related accounts. Both patterns are detected using two paths: native SQL and an equivalent Cypher query through AGE, allowing direct measurement of storage and execution tradeoffs.

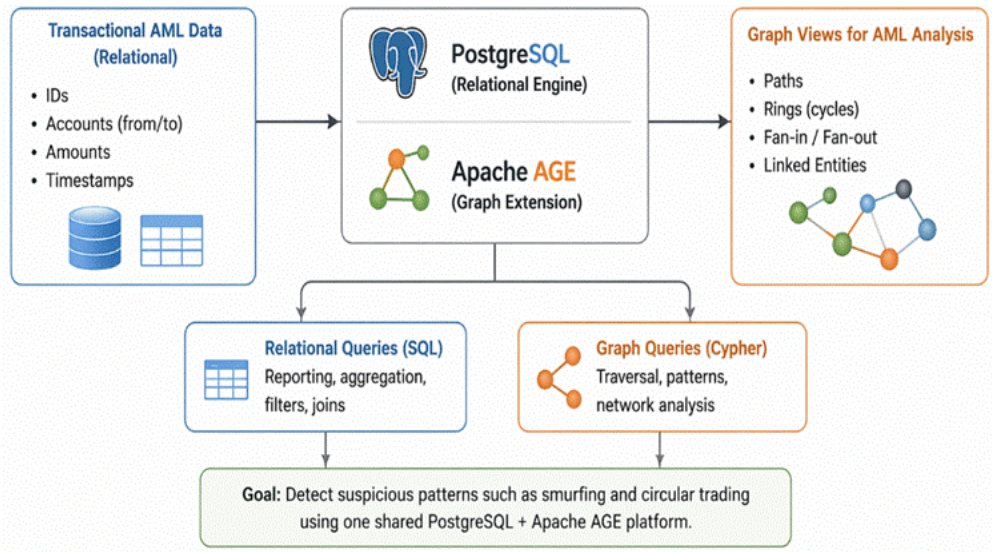

Figure 1: PostgreSQL + Apache AGE as a unified platform for relational and graph AML workloads

The research question is: can PostgreSQL with Apache AGE efficiently support graph traversal workloads compared to native relational approaches, and what are the storage and execution tradeoffs? Results show one stack can support both workloads, but the graph layer introduces measurable overhead from property extraction and additional vertex joins. Findings are grounded in EXPLAIN ANALYZE evidence across 20 warm-cache runs per query on a dataset of 324 customers, 369 accounts, and 1,896 transactions.

Database System Overview

PostgreSQL is an open-source relational DBMS using heap-organized storage where table data lives in 8 KB pages as row versions called tuples. It supports typed columns, composite indexes, recursive queries, and a cost-based query planner. Concurrency is handled through MVCC, allowing readers and writers to operate simultaneously without blocking each other.

Apache AGE is a PostgreSQL extension adding openCypher graph query support without a separate storage engine. Each graph is stored as a namespace in ag_catalog, with every vertex and edge label backed by a PostgreSQL heap table. Properties are stored in agtype format, similar to JSONB. Graph data physically lives in PostgreSQL heap tables sharing the same ACID guarantees, buffer cache, and indexing as relational data.

When a Cypher query is submitted, AGE parses openCypher, transforms it into PostgreSQL's internal query tree, and passes it to the standard planner and executor. The implementation is tested as a Dockerized single-node PostgreSQL + Apache AGE environment using the hybrid_aml database, apache-age container, and host port 5455. Unlike Neo4j, AGE does not use index-free adjacency; unlike MySQL, it supports graph syntax through PostgreSQL parser and planner integration.

Internal Architecture

Storage

The three core relational tables are customers, accounts, and transactions. The transactions table (1,896 rows) is the most query-intensive. Two B-tree indexes directly support fraud detection: a composite index on (from_account_id, tx_time) for outbound transfer lookups, and the primary key on tx_id. The accounts and customers tables use primary key indexes supporting nested loop joins that resolve account ownership and customer names.

Apache AGE stores graph data in ag_catalog. The baseline graph aml_graph contains Account vertices and all TRANSFER edges. A second tuned projection, aml_graph_tuned, contains only transfer-type edges with numeric epoch timestamps, reducing scan cost. Because agtype is not a native PostgreSQL type, every property access requires agtype_access_operator plus an explicit cast on every row, every join condition. This is a primary source of the performance gap between SQL and Cypher paths.

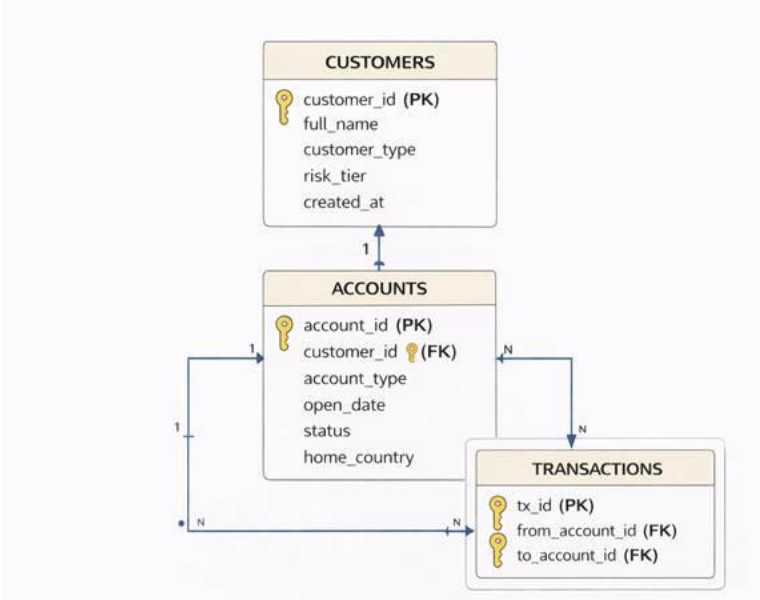

Figure 2: Relational schema showing customers, accounts, and transactions with foreign key relationships

Indexing

B-tree indexes are the default index type in PostgreSQL and the only type used in this project. A B-tree organizes indexed values in a sorted tree structure where each leaf node contains index entries pointing to heap tuple locations. When the query planner determines that an index scan is cheaper than a sequential scan, it traverses the B-tree to find the matching entries and then fetches the corresponding heap pages. For point lookups and small range scans, this is significantly faster than reading every page in the heap file. For queries that would return a large fraction of the table, the planner often prefers a sequential scan because the overhead of random page fetches from an index can exceed the cost of a single linear pass through the heap.

In the smurfing SQL query, this tradeoff appears directly in the EXPLAIN ANALYZE output. The deposit filtering stage performs a sequential scan on the transactions table because it reads all 1,896 rows to find the 105 cash deposit records, filtering out 1,791 rows in the process. At this dataset size, a sequential scan is cheaper than an index lookup because there is no index that efficiently combines tx_type and channel. Once the suspicious deposit clusters are identified, the query switches to an index scan on idx_transactions_from_account_time to find outbound transfers from the flagged account within the funnel window. This is an efficient range scan: the index is sorted by from_account_id first, and then by tx_time, so PostgreSQL can jump directly to the relevant account and scan only the entries within the time range.

In the circular trading recursive SQL query, the planner materializes the filtered transfer edges as a CTE in memory and then performs recursive hash joins entirely in memory, touching only 31 shared buffers. No index is used because the recursive union operates on the in-memory CTE rather than re-accessing the heap for each hop. This is an important observation: for bounded 3-hop cycle detection on this dataset, keeping everything in memory and avoiding index overhead is faster than the graph approach, which performs 792 index searches on TRANSFER_start_id_idx for the first hop alone.

Query Execution

PostgreSQL's pipeline has four stages: parsing, rewriting, planning, and execution. The planner uses table statistics from ANALYZE to select the lowest-cost plan. When AGE receives a Cypher query, it intercepts at the parsing stage, translates openCypher to PostgreSQL's internal query tree, and inserts graph-aware plan nodes. The translated query passes through the standard planner and executor, carrying additional overhead from graph layer translation and agtype extraction on every accessed field.

MVCC and Concurrency

Each tuple carries transaction ID stamps indicating creation and expiry. Readers see only committed versions from their snapshot while writers create new tuple versions, so fraud detection queries do not block ingestion. The prototype is single-node: replication, sharding, and distributed query execution are not implemented. PostgreSQL supports replication, but synchronizing AGE graph projections across replicas would require triggers, rebuilds, or streaming projection work beyond this scope.

Application Design

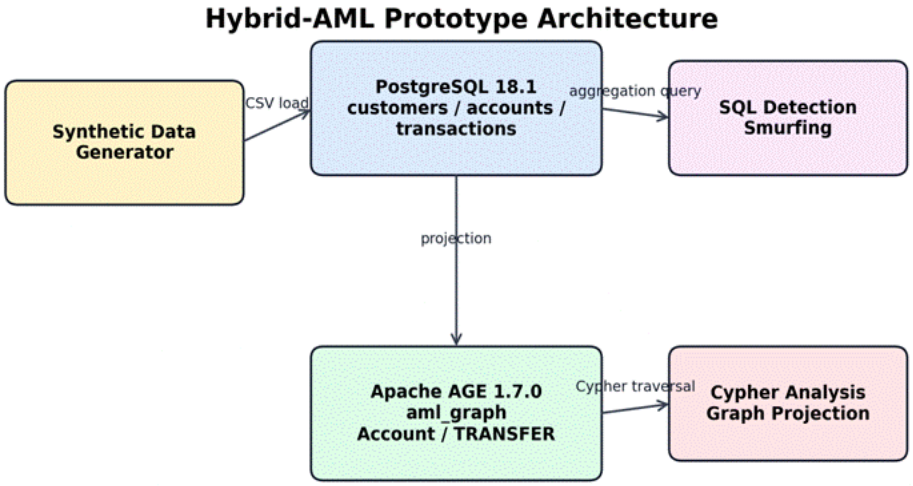

Figure 3: Hybrid-AML prototype architecture connecting synthetic data, relational tables, and AGE graph projection

Hybrid-AML is a command-line, pipeline-based fraud monitoring prototype. Users run reset_pipeline.sh to rebuild data, tables, and AGE graphs; run_demo_queries.sh to inspect detections; smoke_test.sh to verify counts; and benchmark_cycle_queries.py to refresh EXPLAIN ANALYZE artifacts. The design prioritizes reproducibility and separation between SQL and graph paths.

The six components are: (1) a Python generator producing 324 customers, 369 accounts, and 1,896 transactions with nine smurfing and nine circular trading scenarios; (2) relational schema and loader scripts; (3) two AGE graph projections; (4) five SQL/Cypher detection files; (5) a live demo runner; and (6) a benchmark script that runs each query 20 times with EXPLAIN ANALYZE BUFFERS and writes artifacts to docs/reports/.

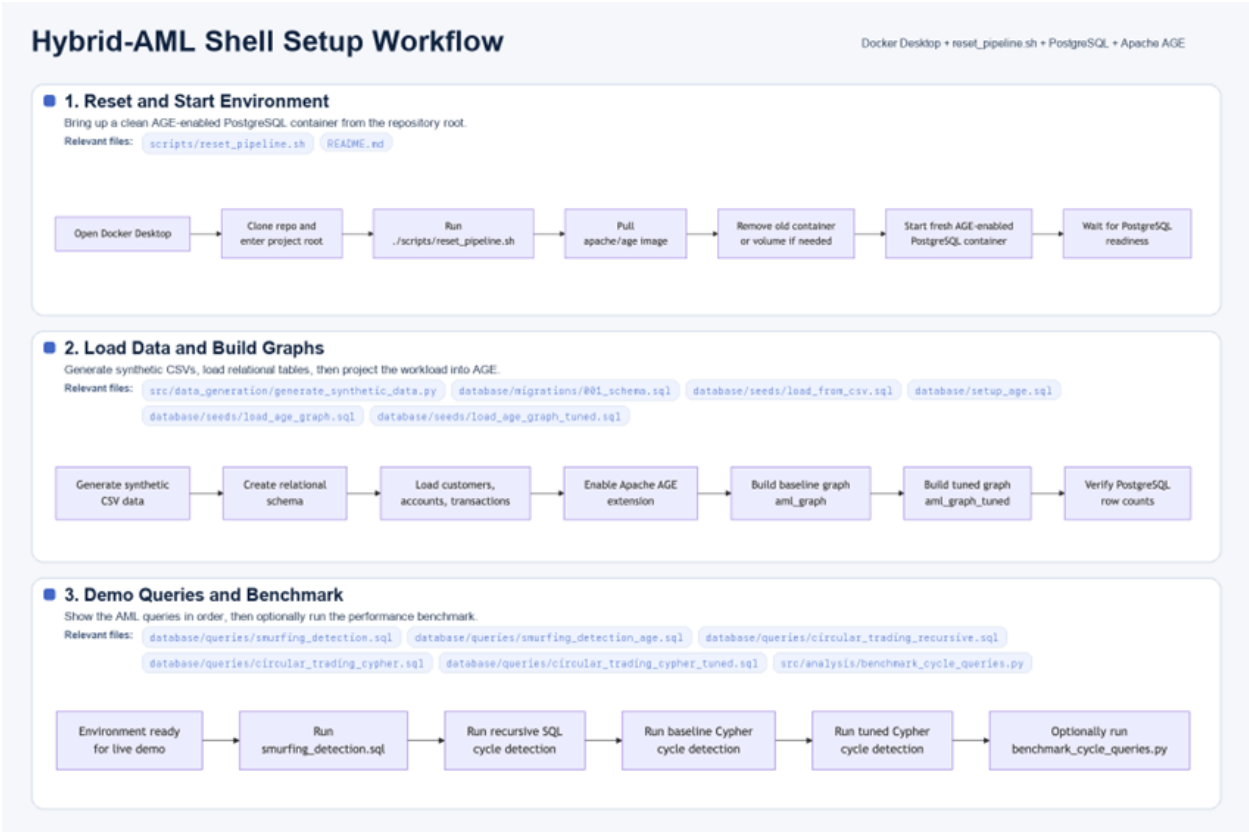

Figure 4: Application setup and execution workflow from reset_pipeline.sh through benchmark

Detection Rules

Smurfing: flags any account receiving five or more cash deposits each below USD 10,000 on the same day totaling at least USD 45,000, followed by an outbound transfer of at least 70% of that total within 24 hours. Representative scenario: ACC-AML-HUB-001 receives six deposits totaling USD 56,800, then wires USD 54,900 to ACC-SHELL-001 on the same day.

Circular trading: flags any three distinct accounts where funds complete a closed A to B to C to A loop in chronological order within six hours. No amount threshold. Representative scenario: ACC-CYCLE-001 sends USD 15,150 to ACC-CYCLE-002 at 10:15, ACC-CYCLE-002 sends USD 14,960 to ACC-CYCLE-003 at 11:03, ACC-CYCLE-003 returns USD 15,040 to ACC-CYCLE-001 at 12:16.

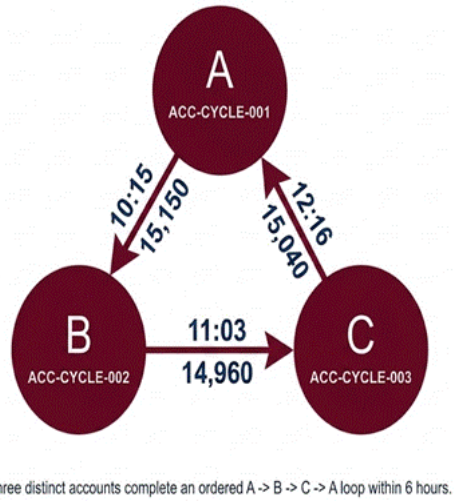

Figure 5: Circular trading scenario showing the A -> B -> C -> A fund flow loop within 6 hours

Mapping Internals to Application Behavior

Operation 1: Smurfing Detection via Relational SQL (0.443ms, 169 buffers)

The query begins with a sequential scan on the public. transactions reading all 1,896 rows to find 105 cash deposits, filtering out 1,791 rows. A sequential scan is chosen because no index combines tx_type and channel. A GroupAggregate groups by beneficiary account and day, removing 51 groups below the threshold and keeping 9 suspicious clusters. For each cluster, an index scan on idx_transactions_from_account_time uses a composite B-tree range scan to find outbound transfers within the 24-hour window, performing 9 index searches. Four nested loops join with accounts_pkey and customers_pkey to resolve customer names. Total: 0.443ms median, 169 shared buffers.

Why it matters: the composite index on (from_account_id, tx_time) reduces 9 potential full table scans to 9 direct B-tree lookups. Without it, the outbound transfer lookup would degrade linearly with dataset size.

Operation 2: Smurfing Detection via Apache AGE (2.503ms, 175 buffers)

AGE executes the same rule on aml_graph.TRANSFER edges. Every property access calls agtype_access_operator and casts from agtype on all 1,896 rows before filtering. After grouping, a Hash Join binds the beneficiary Account vertex, a step the SQL path never needs. Three additional nested loops against the public. accounts and public. Customers recover customer names not projected into the graph. Result: 2.503ms median, 5.7x slower than SQL.

Why it matters: the performance gap has three sources: agtype extraction on every row, an extra Hash Join to bind Account vertices, and relational lookups for attributes not in the graph projection.

Operation 3: Circular Trading Detection via Recursive SQL (1.303ms, 31 buffers)

The query materializes all 792 transfer edges into an in-memory CTE (78 KB) with a single sequential scan. A WITH RECURSIVE union grows 3-hop paths through hash joins on the in-memory CTE, enforcing a 6-hour window and no repeated accounts or transactions. A final CTE scan keeps 9 closed cycles. Total: 1.303ms median, touching only 31 shared buffers because all computation runs against the in-memory CTE after the initial scan.

Why it matters: for bounded 3-hop traversal on a dataset that fits in memory, CTE materialization outperforms index-driven graph traversal because the cost is paid once and amortized across all recursive steps.

Operation 4: Circular Trading Detection via Apache AGE Cypher Baseline (6.97ms, 10,648 buffers)

AGE translates the Cypher MATCH pattern into three separate TRANSFER table scans, one per hop, stitched by hash joins and nested loops. TRANSFER_start_id_idx performs 792 index searches for hop 1 and 477 for hop 2. Agtype extraction applies to every edge property at every hop. _ag_enforce_edge_uniqueness3 adds a filter on each edge. Result: 6.97ms median, 10,648 shared buffers, approximately 5.4x slower than recursive SQL.

Operation 5: Circular Trading Detection via Tuned Cypher (2.586ms, 155 buffers)

The tuned graph aml_graph_tuned contains only transfer-type edges and uses numeric epoch timestamps instead of string timestamps. This eliminates the filter-time agtype string extraction for tx_type on every row and replaces string comparisons with numeric subtraction. Buffer usage drops from 10,648 to 155, execution from 6.97ms to 2.586ms, a 2.7x speedup entirely from projection design. Even after tuning, recursive SQL at 1.303ms remains faster.

Operation	**Key Internal Behavior**	**Median Time**	**Buffers**
Smurfing SQL	Seq scan + GroupAggregate + composite index range scan	0.443ms	169
Smurfing AGE	Seq scan + agtype extraction + Hash Join + index scan	2.503ms	175
Cycle Recursive SQL	CTE materialization + recursive hash joins in memory	1.303ms	31
Cycle Cypher Baseline	3x TRANSFER scans + 792/477 index searches + agtype	6.97ms	10,648
Cycle Cypher Tuned	Pre-filtered graph + hash joins + numeric epoch	2.586ms	155

Table 1: Summary of internal behavior and measured performance for all five detection operations

Comparison with MySQL and MongoDB

This comparison is architectural rather than benchmarked. PostgreSQL + AGE fits this workload because it combines relational integrity with bounded graph traversal. MySQL is stronger for conventional relational OLTP without graph query requirements, while MongoDB is strongest for flexible document storage and sharded document workloads.

vs MySQL

MySQL InnoDB uses a clustered index where the primary key and data share the same B-tree. Every secondary index stores the primary key rather than a direct heap pointer, adding an extra B-tree traversal per matched row. PostgreSQL's composite index on (from_account_id, tx_time) points directly to heap tuples. MySQL also has no native graph extension or AGE-style parser and planner integration. Circular trading detection would require recursive CTEs only, making MySQL a partial fit for the relational portions but not the hybrid graph goal.

vs MongoDB

MongoDB's document model has no native foreign key constraints, so referential integrity between customers, accounts, and transactions must be enforced at the application layer. Multi-document ACID transactions exist but add overhead compared to PostgreSQL's default MVCC. Graph traversal is approximated by $graphLookup, which cannot enforce time-window constraints within traversal or support cycle detection natively. MongoDB scales document workloads well, but it is a weak fit for this relational-plus-graph AML prototype.

Dimension	**PostgreSQL + AGE**	**MySQL**	**MongoDB**
Storage model	Heap + graph labels in ag_catalog	Clustered InnoDB index	BSON document collections
ACID guarantees	Full, default for all ops	Full via InnoDB	Multi-doc transactions, overhead
Graph query support	openCypher via Apache AGE	None	Limited via $graphLookup
Join performance	B-tree, direct heap pointers	B-tree + extra PK lookup	$lookup at aggregation time
Extensibility	Deep parser + planner hooks	Limited plugin system	No equivalent
AML workload fit	Strong	Partial	Weak

Table 2: Comparison of PostgreSQL+AGE, MySQL, and MongoDB across key dimensions

Limitations & Lessons Learned

Limitations

1. Dataset scale: 1,896 transactions fit entirely in PostgreSQL's buffer cache during warm-cache benchmarking. Results reflect best-case conditions. At production scale, sequential scans become bottlenecks, recursive CTEs may spill to disk, and the graph index traversal may become more competitive.
2. Synthetic data: all fraud scenarios are planted at known locations. Real financial data has irregular temporal patterns, evolving account behavior, and fraud embedded in much larger normal activity volumes. False positive rates and edge case handling were not tested.
3. Graph projection synchronization: AGE projections are built once at setup time. Keeping a graph projection current as new transactions arrive would require periodic rebuilds, incremental triggers, or streaming projection mechanisms, none of which are implemented here.
4. Comparison scope: the MySQL and MongoDB comparison is conceptual. A rigorous cross-system benchmark would require identical implementations under identical hardware and workload conditions.

Lessons Learned

1. One stack can support both workloads, but the graph layer has a measurable cost. The graph path is 5.7x slower for smurfing and 5.4x slower for cycle detection using the baseline Cypher. The overhead is concrete: agtype extraction on every row, extra vertex join steps, and repeated index-driven traversal instead of in-memory CTE materialization.
2. Graph projection design matters as much as query design. Moving to aml_graph_tuned reduced buffer usage from 10,648 to 155 and execution from 6.97ms to 2.586ms, a 2.7x speedup from projection design alone, not query rewriting.
3. Recursive SQL is underrated for bounded traversal. The WITH RECURSIVE CTE touches only 31 buffers and completes in 1.303ms, outperforming even the tuned Cypher at 2.586ms. For fixed-depth graph patterns on moderate datasets, recursive SQL is a strong alternative to a graph query layer.
4. Physical storage format determines performance more than query language. Cypher is more concise than recursive SQL, but conciseness does not translate to faster execution when storage requires a type extraction and B-tree traversal on every hop. A more expressive query language does not guarantee a more efficient execution plan.

Project Links

GitHub Repository: https://github.com/KHUCHAN/DSCI-551-Data-Base-Course-Project.git

The repository contains the source code, schema, setup scripts, synthetic data generator, detection queries, benchmark scripts, EXPLAIN ANALYZE artifacts, and documentation. Key paths include README.md for setup, scripts/reset_pipeline.sh for one-command setup, scripts/run_demo_queries.sh for detections.